

第 6 章

MCPの 実運用に向けて

Model Context Protocol

6.1 MCPサーバーと連携したAIエージェントの評価

6.1.1 概要

MCPサーバーとAIエージェントが連携することで、AIエージェントにさまざまな機能を簡単に持たせることができます。一方で、AIエージェントがMCPサーバーが提供するさまざまなツールを適切に利用できたかは、入出力結果からは単純に判断できません。AIエージェントが選択したツールやその引数が開発者が想定したものかを、定量的に評価します。AIエージェントのツール利用結果を評価することで、AIエージェントが持つプロンプトやdocstringで記載されたMCPサーバーが提供するツールの説明テキストを改善するために役立てることができます。

本節では、四則計算をする簡単なMCPサーバーに対して評価処理を追加して、AIエージェントが期待どおりにMCPサーバーが提供するツールを利用できたかを確認する方法をハンズオンを通して紹介します。

6.1.2 LLMアプリケーションにおける評価

AIエージェントを含むLLMアプリケーションの評価は、開発段階からサービス提供開始後の運用段階にいたるまで、性能を改善・維持するために重要です。しかしながら、非決定的に動作するLLMの出力は実行ごとに変化します。LLMの出力内容やその指示である入力も自然言語であるケースが多いため、単純に単語やその並びを比較するだけでは正しく評価できず、「出力の意味」を比較する必要があります。人は入力や出力であるテキストの意味を考慮できますが、自然言語処理で用いられてきたBLEUやROUGEという評価指標では意味を考慮しないため、評価が困難です。

■自然言語の比較評価

単語の並びが異なるから別の意味だ!

今日は不安定な空模様です

今日の天候は変わりやすいです

単語の並びが異なるけど同じ意味だ!

人が入出力テキストをチェックすることで、テキストの意味を考慮した評価ができます。しかしながら、実行結果を人が評価する場合、作業時間と費用が膨大になるため、多くのテストケースを評価することが難しくなります。これらの問題を解決するために、LLM-as-a-Judgeという評価手法が用いられるケースが増えています。

LLM-as-a-Judgeは、評価基準を定義したプロンプト（以下、評価プロンプト）をもとにLLMが評価する手法です。プロンプトには何を、どのような観点で評価し、評価値をどのように出力するかを記載します。

与えられたルールに従って、ユーザー入力とLLM回答の関連性を評価してください。

関連性を評価する際の基準

- LLM回答がユーザー入力に直接関係している場合、関連性は高いと判断する
- LLM回答の中に、ユーザー入力に関連しない内容が含まれている場合、関連性は低いと判断する

関連性の評価は以下のいずれかにしてください。

- weak : ユーザー入力とLLM回答のほとんど関連していない
- moderate: ユーザー入力とLLM回答がある程度関連している
- strong : ユーザー入力とLLM回答が強く関連している

出力形式は以下のJSONスキーマに従ってください。
キーはそのままとして、値のみ置き換えてください。

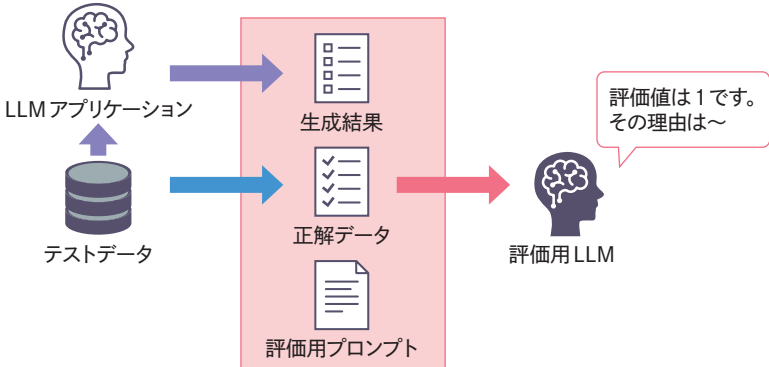
```
{  "score": [ "weak", "moderate", "strong" ],  "reason": "<評価結果の理由>"}
```

<ユーザー入力>
{input}>
<LLM回答>
{output}>

評価基準を定義した評価プロンプトと、実際の入出力結果をLLMに渡して評価します。LLMは、自然言語の意味も参考にして評価でき、スクリプトなどから自動で呼び出すこともできるので、評価プロセスのコストを大きく下げることができます。

事前に入力データと正解データをテストデータセットとして用意し、LLMの出力データと正解データをLLM-as-a-Judgeで評価することもできます。テストデータセットを用意し、出力データを評価する方法をオフライン評価と呼びます。これに対して、本番環境での実データを用いた評価をオンライン評価と呼びます。オンライン評価では正解データを用意できないため、先ほどの評価プロンプトのように、入力と出力の関連性やハルシネーション検出などを評価します。

■ LLM-as-a-Judge による評価



LLM-as-a-Judge にも以下のような課題があります。

- 評価にバイアスが入り込む可能性がある
- 評価プロンプト自体の品質を管理する必要がある

評価プロセスに LLM が介在するため、評価値に LLM によるバイアスが入り込む可能性があります。人の感覚とは異なる評価をする場合もあるので注意が必要です。評価結果に加えて「なぜそのように判断したか」を出力することで、判断根拠を確認するとよいでしょう。

LLM-as-a-Judge は、評価プロンプトをもとに評価します。評価プロンプトに記載した評価基準が不明瞭であると、意図した観点で評価されない、評価ごとに結果が大きく揺らぐなど評価結果そのものを信頼できなくなります。そのため評価プロンプト自体の品質を管理する必要があります。

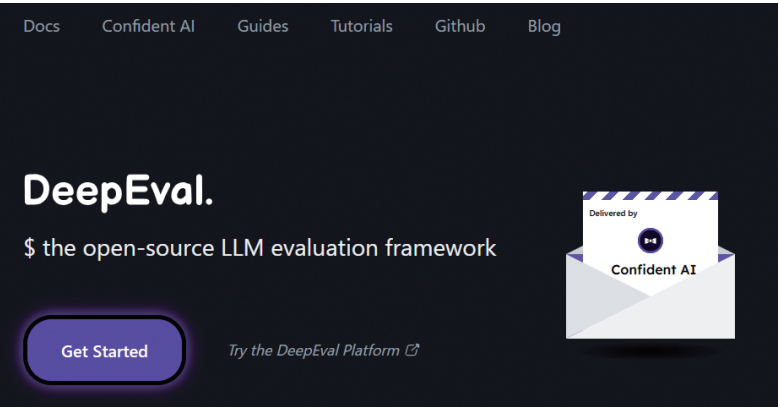
LLM-as-a-Judge は、幅広い LLM アプリケーションに適用でき、自動化も可能です。考慮すべき点がありますが、評価プロンプトを用意すればすぐに試すことができるので、開発の初期段階から導入し、LLM アプリケーションと共に評価プロンプトの品質を高めるとよいでしょう。

6.1.3 ■ LLM-as-a-Judge に挑戦しよう

前項では LLM アプリケーションを評価する手法として LLM-as-a-Judge を紹介しました。本項では評価フレームワークである DeepEval を利用して、LLM-as-a-Judge による評価に挑戦してみます。

DeepEval^{注6.1} は Confident AI が開発しているオープンソースの LLM 評価フレームワークです。DeepEval ではさまざまな LLM アプリケーションのユースケースで利用可能な評価メトリクスを提供しています。Pytest のように LLM アプリケーションの評価テストをしたり、テスト用データセットの生成を支援したりする機能もあります。

■ DeepEval 公式サイト



DeepEval を用いた AI エージェントの評価フローは次となります。

■ DeepEval による AI エージェントの評価フロー



「テストデータの準備」では、評価に用いる入出力データを用意します。さまざまなケースを網羅するためには多様なテストデータが必要ですが、質の良いテストデータをそろえるのは簡

注6.1 <https://deepeval.com/>

単ではありません。そのため、テストデータ数が少なくても、まず評価を始めて性能を可視化することが重要です。

「AIエージェントの実行」では、テストデータを用いてAIエージェントを実行します。最終出力だけでなく、**AIエージェントの行動履歴である軌跡**も取得する必要があります。

「テストケースの定義」では、評価の対象となるAIエージェントの入出力、軌跡などの実行結果を一つのクラスインスタンスにまとめます。テストケースとして、単一のインタラクションを表現した**シングルターン**と複数回のインタラクションを表現した**マルチターン**が提供されています。評価したい内容に合わせていずれかを選択します。

「評価メトリクスの選択」は、AIエージェントの評価観点を決定します。DeepEvalでは、RAGやAIエージェント、安全性検証などさまざまな評価メトリクスが事前定義されています。事前定義された評価メトリクスを利用することで、評価プロンプトや評価アルゴリズムを実装することなく評価できます。独自の評価メトリクスを定義するためのフレームワークも用意されているので、評価メトリクスを自身で用意することも可能です。

最後に用意したテストケースと評価メトリクスを用いて、評価を実行します。

それではDeepEvalを用いて、LLM-as-a-Judgeに挑戦しましょう。本ハンズオンはGitHub Codespaces を使用し実施します。以下の準備を行ってから読み進めてください。

■ ハンズオンの事前準備

準備項目	参照先
AWSアカウントの作成とIAMユーザーの作成	付録A.1 AWSのセットアップ
Bedrock呼び出しの準備	付録A.2 Bedrockのユースケース送信とクォータの引き上げ
GitHub Codespaces環境の初期設定	付録A.3 GitHub Codespaces環境構築

本ハンズオン向けにuvを用いてPythonプロジェクトを作成します。下記コマンドをCodespacesのターミナル上で実行してください。

```
$ cd /workspaces/mcp-aws-handbook
$ mkdir -p chapter6
$ cd chapter6
$ uv init llm-as-a-judge --python 3.13
```

作成されたllm-as-a-judgeディレクトリの下にeval.pyを作成します。初期化時に作成されたmain.pyとREADME.mdは使用しないため、削除してください。

```
/workspaces/mcp-aws-handbook/chapter6/
└─ llm-as-a-judge
    └─ .python-version
```

```
└─ eval.py
└─ pyproject.toml
└─ uv.lock
```

ハンズオンに必要なパッケージをインストールします。次のコマンドを実行して、DeepEvalと依存パッケージをインストールします。

```
$ cd llm-as-a-judge
$ uv add deepeval==3.7.0 aiobotocore==2.25.1
```

LLM-as-a-Judgeの実装に移ります。作成したeval.pyにこれから紹介するスクリプトを記載してください。はじめにDeepEvalと非同期実行するために必要なasyncioパッケージをインポートします。

chapter6/llm-as-a-judge/eval.py (1/6)

```
import asyncio
from deepeval.metrics import GEval
from deepeval.test_case import LLMTestCaseParams
from deepeval.test_case import LLMTestCase
from deepeval.models import AmazonBedrockModel
from deepeval import evaluate
```

eval_relevance関数はここから紹介する評価に必要な一連の処理を担う関数です。最初にLLM-as-a-Judgeで利用する評価用LLMを定義します。評価用LLMとして、Bedrockを利用するため、DeepEvalが提供するAmazonBedrockModelクラスのインスタンスを生成しています。

chapter6/llm-as-a-judge/eval.py (2/6)

```
async def eval_relevance():
    # 評価モデルを初期化
    deepeval_model = AmazonBedrockModel(
        model_id="us.anthropic.claude-haiku-4-5-20251001-v1:0",
        region_name="us-west-2"
    )
```

LLM-as-a-Judgeの評価メトリクスを定義します。DeepEvalでは事前定義されたメトリクスが多く提供されていますが、開発者自身が評価基準を設定する機能も提供されています。GEvalクラスでは、evaluation_stepsに評価基準をリスト形式で渡すことで、CoT (Chain-of-Thought)を用いて評価します。今回はユーザー入力とLLMの出力の関連性を評価するように指示しています。

evaluation_paramsでは評価に必要な情報を指定します。今回は入力とLLMの出力を指定しています。modelはLLM-as-a-Judgeで使用する評価用LLMです。

chapter6/llm-as-a-judge/eval.py (3/6)

```
# 評価メトリクスの設定
metric = GEval(
    name="関連性チェック",
    evaluation_steps=[
        "LLMの回答がユーザー入力に直接関係している場合、関連性は高いと判断する",
        "LLMの回答の中に、ユーザー入力に関連しない内容が含まれている場合、関連性は低いと判断する",
    ],
    evaluation_params=[LLMTestCaseParams.INPUT, LLMTestCaseParams.ACTUAL_OUTPUT],
    model=deepeval_model
)
```

テストケースを作成します。以下では、評価に必要なユーザー入力とLLM出力を引数として渡しています。今回はLLM出力をする引数actual_outputに文字列を直接入力しています。実運用ではここにLLMの出力結果を入力してください。

chapter6/llm-as-a-judge/eval.py (4/6)

```
# テストケースの設定
test_case = LLMTestCase(
    input="今日の天気を教えてください？",
    actual_output="今日の天気は晴れです。外出しやすく洗濯日和でしょう。",
)
```

eval_relevance関数の最後では、evaluate関数を呼び出して評価を実施します。引数として、ここまでに生成したテストケースとメトリクスを指定します。

chapter6/llm-as-a-judge/eval.py (5/6)

```
evaluate(test_cases=[test_case], metrics=[metric])
await deepeval_model.close()
```

最後にエントリポイントを定義します。

chapter6/llm-as-a-judge/eval.py (6/6)

```
if __name__ == "__main__":
    asyncio.run(eval_relevance())
```

それでは、Pythonスクリプトを実行して評価結果を確認しましょう。以下のコマンドをCodespacesのターミナルから実行してください。

```
$ uv run eval.py
```

実行結果例は下記となります。結果は実行ごとに変わるためご注意ください。

実行結果例

```
~中略~
=====

Metrics Summary

-  関連性チェック [GEval] (score: 0.9, threshold: 0.5, strict: False, evaluation model: us.anthropic.claude-haiku-4-5-20251001-v1:0, reason: LLMの回答は、ユーザー入力「今日の天気を教えてくださいですか？」に直接関係しており、高い関連性を示しています。天気情報(晴れ)を提供し、さらに関連する有用な情報(外出しやすさ、洗濯日和)を付加しています。ユーザー入力に関連しない内容は含まれていません。わずかに減点した理由は、実際の天気データソースが明示されていない点です。 error: None)

For test case:

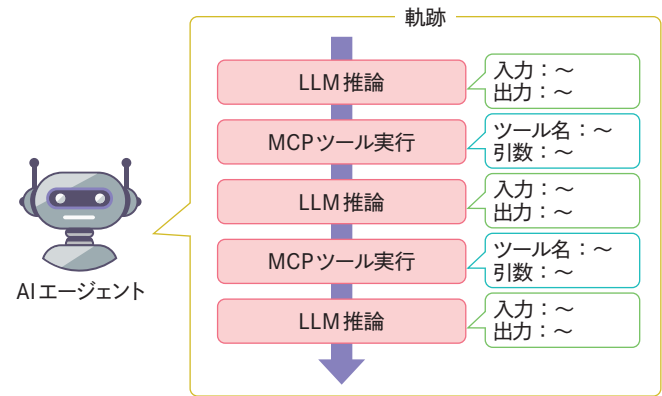
- input: 今日の天気を教えてください？
- actual output: 今日の天気は晴れです。外出しやすく洗濯日和でしょう。
- expected output: None
- context: None
- retrieval context: None
~中略~
```

結果から指定した評価基準に従って、ユーザー入力と回答結果の関連性を評価できたことがわかります。今回の例のように、LLM-as-a-Judgeを活用することで、と出力である自然言語の意味を考慮した評価が可能です。

6.1.4 MCPサーバーを活用するAIエージェントの評価方法

MPCサーバーを使うAIエージェントの評価方法について紹介します。AIエージェントはユーザーの要求に応じて、から適切なツールを選択して利用します。そのためAIエージェントの入出力を評価するだけでなく、最終出力にいたるまでにどのような行動をしたかを示す軌跡を含めて評価する必要があります。

AIエージェントの軌跡



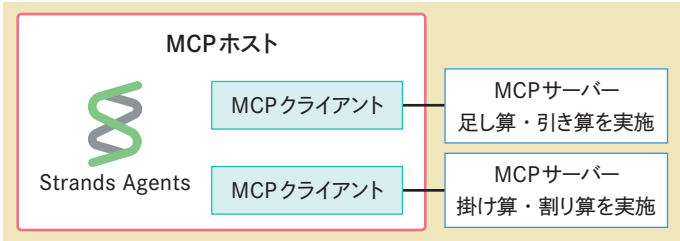
本節のハンズオンではこの軌跡を用いて、AIエージェントが連携するMCPサーバーが提供するツールを、正しく利用したかを評価します。AIエージェントの軌跡を評価するアルゴリズムを一から実装するのは簡単ではありません。そのため、アルゴリズムや評価プロンプトを定義し、評価指標として提供している評価フレームワークを利用します。評価フレームワークを利用することで、事前準備なしで手軽に評価できます。

DeepEvalでは、MCPサーバーと連携したAIエージェントを評価する際に利用可能なメトリクスである「MCP-Useメトリクス」を提供しており、今回はこの評価メトリクスを用います。MCP-Useメトリクスは、AIエージェントがMCPサーバーが提供するプリミティブを効果的に利用して回答が生成できたか評価します。

6.1.5 ■ AIエージェントを評価してみよう

それではDeepEvalを用いてMCPサーバーと連携したAIエージェントを評価しましょう。本ハンズオンでは、足し算と引き算を実施するMCPサーバーと掛け算と割り算をするMCPサーバーをそれぞれ用意し、それらと連携するAIエージェントをDeepEvalで評価します。

評価対象となるAIエージェント



本ハンズオンでのファイル構成は以下のとおりです。

```
/workspaces/mcp-aws-handbook/chapter6/
├── eval-agent
│   ├── .python-version
│   ├── addsub_server.py
│   ├── calc_agent.py
│   ├── muldiv_server.py
│   ├── pyproject.toml
│   └── uv.lock
```

◎ 事前準備

本ハンズオンはGitHub Codespacesを使用し実施します。以下の準備を行ってから読み進めてください。

ハンズオンの事前準備

準備項目	参照先
AWS アカウントの作成とIAMユーザーの作成	付録A.1 AWSのセットアップ
Bedrock呼び出しの準備	付録A.2 Bedrockのユースケース送信とクォータの引き上げ
GitHub Codespaces環境の初期設定	付録A.3 GitHub Codespaces環境構築

はじめにuvを用いてプロジェクトを作成します。下記コマンドをCodespacesのターミナルで実行してください。

```
$ cd /workspaces/mcp-aws-handbook
$ mkdir -p chapter6
$ cd chapter6
$ uv init eval-agent --python 3.13
```

作成したeval-agentディレクトリ以下にあるmain.pyとREADME.mdは使用しないため、削除してください。

AIエージェントとDeepEvalを動作させるために必要なPythonパッケージをインストールします。Codespacesのターミナルを開き、次のコマンドを実行してください。

```
$ cd eval-agent
$ uv add mcp==1.21.0 strands-agents==1.15.0 deepeval==3.7.0 aiobotocore==2.25.1
```

◎ MCP-Useメトリクス

AIエージェントを評価するためのメトリクスとして、MCP-Useメトリクスを利用します。MCP-Useメトリクスは、MCPサーバーが提供するプリミティブをAIエージェントが正しく、効率よく利用できているかを評価するメトリクスです。どのようなプリミティブを利用したかに加えて、ツール呼び出し時の引数もチェックします。

MCP-Useメトリクスは、ユーザー入力に対して、AIエージェントが回答する単一のインタラクションを評価します。複数のインタラクション（マルチターン）向けには、Multi-Turn MCP-Useメトリクスが提供されています。

MCP-Useメトリクスを利用するために必要な入力データは以下のとおりです。これらのデータをテストケースとして、定義します。

■ MCP-Useメトリクス利用に必要なデータ

データ	概要
AIエージェントへの入力	AIエージェントに入力するデータ
AIエージェントの出力	AIエージェントの最終回答
MCPサーバーが提供するプリミティブ	AIエージェントが利用できるMCPサーバーとそのプリミティブプリミティブにはツール、プロンプト、リソースが指定可能
AIエージェントが利用したMCPサーバーのプリミティブ	AIエージェントが回答生成するために実際に利用したMCPサーバーのプリミティブ

◎ MCPサーバーの実装

はじめにAIエージェントと連携するMCPサーバーを実装します。実装するMCPサーバーは、足し算と引き算機能をツールとして提供するMCPサーバー、掛け算と割り算機能をツールとして提供するMCPサーバーの2つです。2つのMCPサーバーはいずれもstdio方式でMCPクライアントと接続します。

まず足し算と引き算機能を提供するMCPサーバーを実装します。addsub_server.pyを作成し、以下のスクリプトを記載してください。

■ chapter6/eval-agent/addsub_server.py

```
from mcp.server.fastmcp import FastMCP

# FastMCP サーバーを初期化
mcp = FastMCP("AddSubServer")

@mcp.tool()
def add(a: int, b: int) -> int:
```

注6.2 <https://deepeval.com/docs/metrics-mcp-use>

```
"""
2つの整数を足し算して結果を返す
Args:
    a: 第1の整数
    b: 第2の整数
"""
return a + b

@mcp.tool()
def sub(a: int, b: int) -> int:
    """
    第1の整数から第2の整数を引き算して結果を返す
    Args:
        a: 第1の整数
        b: 第2の整数
    """
    return a - b

def main():
    """サーバーを実行"""
    mcp.run()

if __name__ == "__main__":
    main()
```

このMCPサーバーは足し算を実施するadd関数と引き算を行うsub関数をMCPのツールとして提供します。

もう一つのサーバーも実装します。muldiv_server.pyという名前のファイルを作成し、以下のスクリプトを記載してください。muldiv_server.pyは掛け算と割り算をツールとして提供するMCPサーバーの実装です。

■ chapter6/eval-agent/muldiv_server.py

```
from mcp.server.fastmcp import FastMCP

# FastMCP サーバーを初期化
mcp = FastMCP("MulDivServer")

@mcp.tool()
def mul(a: int, b: int) -> int:
    """
    2つの整数を掛け算して結果を返す
    Args:
        a: 第1の整数
        b: 第2の整数
    """
    return a * b
```

```
@mcp.tool()
def div(a: int, b: int) -> float:
    """
    第1の整数を第2の整数で割り算して結果を返す
    Args:
        a: 第1の整数
        b: 第2の整数
    """
    if b == 0:
        raise ZeroDivisionError("0で割ることはできません")
    return a / b

def main():
    """サーバーを実行"""
    mcp.run()

if __name__ == "__main__":
    main()
```

◎ AIエージェントと評価機能の実装

それではAIエージェントと評価機能を実装しましょう。eval-agentディレクトリ以下にcalc_agent.pyを作成し、これから紹介するスクリプトを実装してください。

はじめにAIエージェントと評価に必要なPythonパッケージをインポートします。AIエージェントのフレームワークとして利用するStrands Agentsと評価で利用するDeepEvalのパッケージとなります。

chapter6/eval-agent/calc_agent.py (1/8)

```
import asyncio
import os
from mcp.client.stdio import stdio_client, StdioServerParameters
from strands import Agent
from strands.models import BedrockModel
from strands.tools.mcp.mcp_client import MCPClient
from mcp.types import CallToolResult, TextContent
from typing import Dict, List
# DeepEvalによる評価で利用
from deepeval.test_case import LLMTestCase, MCPServer, MCPToolCall
from deepeval.metrics import MCPUseMetric
from deepeval.models import AmazonBedrockModel
from deepeval import evaluate
```

次にAIエージェントを定義するCalcAgentを定義します。CalcAgentクラスのコンストラクタでは、MCPサーバーと連携するために、MCPクライアントを生成します。2つのMCPサーバー

と接続するため、MCPクライアントも2つ生成します。

create_stdio_mcp_client関数は、stdio方式でMCPサーバーと接続するMCPクライアントを生成する関数です。コンストラクタから利用します。

chapter6/eval-agent/calc_agent.py (2/8)

```
# 数値計算エージェントクラス
class CalcAgent:
    def __init__(self):
        # 足し算、引き算するMCPサーバーとのクライアントを生成
        self.addsub_mcp_server = self.create_stdio_mcp_client(
            command="python",
            args=["./addsub_server.py"],
        )
        # 掛け算、割り算するMCPサーバーとのクライアントを生成
        self.muldiv_mcp_server = self.create_stdio_mcp_client(
            command="python",
            args=["./muldiv_server.py"],
        )

    def create_stdio_mcp_client(
        self, command: str, args: List[str], env: Dict = {}
    ) -> MCPClient:
        """stdio MCPクライアントを作成する関数"""
        stdio_mcp_client = MCPClient(
            lambda: stdio_client(
                StdioServerParameters(command=command, args=args, env=env)
            ),
            startup_timeout=60,
        )
        return stdio_mcp_client
```

create_agent関数は、受け取ったツールを利用するAIエージェントを構築します。システムプロンプトでMCPサーバーを活用するように指示しています。

chapter6/eval-agent/calc_agent.py (3/8)

```
def create_agent(self, tools: list) -> Agent:
    """利用可能なツールを使ってエージェントを作成"""
    system_prompt = (
        "MCPサーバーを活用して、ユーザーの質問に回答してください"
    )
    # Bedrockのモデルを定義
    model = BedrockModel(model_id="us.anthropic.claude-haiku-4-5-20251001-v1:0")
    return Agent(
        model=model,
        system_prompt=system_prompt,
```



```
tools=tools,
callback_handler=None,
)
```

ユーザー入力をもとに、AIエージェントを呼び出す invoke 関数を実装します。invoke 関数では、2つの MCP サーバーと接続し、提供されるツールのリストを取得します。その後、AI エージェントを呼び出して回答を生成し、実行結果と MCP サーバーのツールリストを戻り値とします。ツールリストは MCP サーバーごとに返します。

chapter6/eval-agent/calc_agent.py (4/8)

```
async def invoke(self, query: str):
    """ 質問に対して回答を生成し、使用したツール情報も返す """
    with self.addsub_mcp_server, self.muldiv_mcp_server:
        # 各MCPクライアントから利用可能なツールを取得
        addsub_mcp_tools = self.addsub_mcp_server.list_tools_sync()
        muldiv_mcp_tools = self.muldiv_mcp_server.list_tools_sync()

        # 全てのツールを組み合わせてエージェントを作成・実行
        agent = self.create_agent(tools=addsub_mcp_tools + muldiv_mcp_tools)
        response = agent(query)
        return response, addsub_mcp_tools, muldiv_mcp_tools
```

DeepEval のテストケースは、AI エージェントのツール実行履歴を、DeepEval が提供する MCPTool クラスのリストで定義する必要があります。MCPTool クラスは、ツール名、呼び出し時の引数、ツールの実行結果の3つの情報を保持します。ツールの実行結果は MCP Python SDK が定義する CallToolResult クラスである必要があります。Strands Agents の実行結果から DeepEval のテストケースを作成するために、以下の3つの関数を通してツール実行結果の抽出とデータ形式を変換します。

データ変換ユーティリティ関数

関数名	入力	出力	処理内容
extract_tool_results	Strands Agents の実行結果	MCPToolClass のリスト	実行結果の1アクションごとに collect_tools 関数と convert_tool_results 関数を呼び出す
collect_tools	Strands Agents の実行結果 (1アクション)	ツール入出力情報をまとめた辞書変数	Strands Agents の実行結果からツール実行情報を抽出する
convert_tool_results	ツール入出力結果をまとめた辞書変数	MCPToolClass のリスト	ツール出力を CallToolResult に変換し、MCPToolClass に格納する

3つの関数の実装スクリプトは以下となります。

chapter6/eval-agent/calc_agent.py (5/8)

```
def extract_tool_results(data):
    """ エージェント実行データからツール使用結果を抽出 """
    results = []
    for cycle in data.get("traces", []):
        tools = collect_tools(cycle)
        results.extend(convert_tool_results(tools))
    return results

def collect_tools(cycle):
    """ 実行サイクルからツール情報を収集 """
    tools = {}

    for child in cycle.get("children", []):
        if not child or not child.get("message"):
            continue

        for content in child["message"].get("content", []):
            # ツール使用の記録
            if "toolUse" in content:
                tool_use = content["toolUse"]
                tools[tool_use["toolUseId"]] = {
                    "name": tool_use["name"],
                    "input": tool_use.get("input", {}),
                    "output": None,
                }
            # ツール結果の記録
            elif "toolResult" in content:
                tool_result = content["toolResult"]
                tool_id = tool_result["toolUseId"]
                if tool_id in tools:
                    result_content = tool_result.get("structuredContent", {})
                    tools[tool_id]["output"] = result_content.get("result")

    return tools

def convert_tool_results(tools):
    """ ツール情報をMCPToolCall形式に変換 """
    results = []

    for tool in tools.values():
        if not (tool["name"] and tool["output"]):
            continue

        # CallToolResultオブジェクトを作成
        call_tool_result = CallToolResult(  

```

```
        content=[TextContent(type="text", text=str(tool["output"])),
        isError=False,
    )

    # 構造化データがある場合は追加
    if isinstance(tool["output"], dict):
        call_tool_result.structuredContent = tool["output"]

    # MCPToolCallオブジェクトを作成
    results.append(
        MCPToolCall(
            name=tool["name"],
            args=tool["input"],
            result=call_tool_result,
        )
    )

    return results
```

ここまでが評価対象となるAIエージェントの実装です。ここからは実装したAIエージェントを呼び出し、その結果を評価する機能を実装します。

get_deepeval_mcp_server関数はDeepEvalのテストケースに必要なMCPサーバー情報を生成します。関数の引数として、MCPサーバー名とツールリストを受け取り、それを用いてMCPServerクラスを生成します。

```
chapter6/eval-agent/calc_agent.py (6/8)

def get_deepeval_mcp_server(server_name, tools):
    """評価用のMCPServerオブジェクトを作成"""
    return MCPServer(
        server_name=server_name,
        available_tools=[t.mcp_tool for t in tools],
    )
```

eval_mcp_use関数では評価を実施します。まずテストケースを作成します。LLMTestCaseクラスはDeepEvalが提供するシングルターン用のテストケースです。ユーザーからの入力、AIエージェントの最終回答、MCPサーバーのリスト、ツールの実行結果を入力し、LLMTestCaseクラスを初期化します。

MCP-Useメトリクスは、LLM-as-a-Judgeによる評価値計算のため、評価用LLMを指定する必要があります。評価用LLMにはAmazon Bedrockを用いるため、AmazonBedrockModelクラスのインスタンスを定義します。

MCP-UseメトリクスのクラスであるMCPUseMtricsを定義します。今回指定する引数の詳細

は下表のとおりです。その他のオプション引数は公式サイトを参考にしてください^{注6.3}。

MCPUseMetricの引数	
引数名	概要
threshold	評価テストが成功となるしきい値
model	評価に用いるLLM
include_reson	評価スコアの判断記入を出力するかどうか

最後に evaluate 関数を呼び出して、評価を実施します。

```
chapter6/eval-agent/calc_agent.py (7/8)

async def eval_mcp_use(input, output, mcp_servers, call_tool_results):
    """MCPツールの使用状況の評価"""

    # テストケースを作成
    test_case = LLMTestCase(
        input=input,
        actual_output=output["content"][0]["text"],
        mcp_servers=mcp_servers,
        mcp_tools_called=call_tool_results,
    )

    # 評価モデルを初期化
    deepeval_model = AmazonBedrockModel(
        model_id="us.anthropic.claude-haiku-4-5-20251001-v1:0",
        region_name="us-west-2"
    )

    # MCPの使用状況の評価
    mcp_use_metrics = MCPUseMetric(threshold=0.8, model=deepeval_model, include_reason=True)
    evaluate(test_cases=[test_case], metrics=[mcp_use_metrics])
    await deepeval_model.close()
```

evaluateAgent関数ではここまで紹介した関数を順次実行します。今回は1件のテストデータに対して、AIエージェントを評価しています。

```
chapter6/eval-agent/calc_agent.py (8/8)

def evaluateAgent():
    """メイン実行関数：RAGエージェントの動作例とMCP評価のデモ"""

    # RAGエージェントを初期化
    calc_agent = CalcAgent()

    # テスト用クエリ
```

注6.3 <https://deepeval.com/docs/metrics-mcp-use>

```
query = "3+4+5/5="

# エージェントを実行して回答と使用ツール情報を取得
response, addsub_mcp_tools, muldiv_mcp_tools = asyncio.run(calc_agent.invoke(query))

# ツール実行結果を抽出
call_tool_results = extract_tool_results(response.metrics.get_summary())

# 評価用のMCPサーバーを作成
mcp_servers = [
    get_deepeval_mcp_server("addsub_mcp_server", addsub_mcp_tools),
    get_deepeval_mcp_server("muldiv_mcp_server", muldiv_mcp_tools),
]

# MCPツールの使用状況の評価
asyncio.run(eval_mcp_use(query, response.message, mcp_servers, call_tool_results))

if __name__ == "__main__":
    evaluateAgent()
```

◎評価の実行

それでは実装したスクリプトを実行して、AIエージェントを評価します。Codespacesのターミナルから以下のコマンドを実行しています。

```
$ uv run calc_agent.py
```

実行例は以下のようになります。Metrics Summaryで指定したメトリクスの評価スコアとその判断根拠が表示されます。評価スコアは1.0となっており、理想どおりに動作できていることがわかります。判断根拠からも適切なツールを適切な順番で呼び出しているかを正確に判断できていることがわかります。

■実行結果例

```
🌟 You're running DeepEval's latest MCP Use Metric! (using us.anthropic.claude-haiku-4-5-2025-1001-v1:0, strict=False, async_mode=True)...

=====

Metrics Summary

- ✅ MCP Use (score: 1.0, threshold: 0.8, strict: False, evaluation model: us.anthropic.claude-haiku-4-5-20251001-v1:0, reason: [
    The agent correctly selected and used all appropriate tools for the mathematical expression 3+4+5/5. Following proper order of operations, the agent: (1) used 'div' to calculate 5/5=1.0, (2) used 'add' to calculate 3+4=7, and (3) used 'add' to calculate 7+1=8.
  ]
)
```

```
All three tools chosen (div, add, add) were necessary and correct, with each operation applied in the proper sequence. No unnecessary tools were called, and no better tool options were available from the provided primitives.

All arguments passed to the tools were appropriate and correct. The agent correctly decomposed the expression 3+4+5/5 following order of operations: (1) div(5, 5) → 1.0, (2) add(3, 4) → 7, (3) add(7, 1) → 8. Each tool received the correct integer arguments matching their expected input schema (a and b as integers), and the values were semantically appropriate for computing the final result of 8. No required arguments were missing, no extraneous arguments were included, and all argument types matched the tool expectations.
]
, error: None)
~中略~

✓ Evaluation completed 🎉! (time taken: 5.88s | token cost: 0.0 USD)
» Test Results (1 total tests):
  » Pass Rate: 100.0% | Passed: 1 | Failed: 0

=====
~中略~
***評価結果の根拠(Reason)の日本語訳***
エージェントは、数式3+4+5/5に対して、すべての適切なツールを正しく選択して使用しました。演算子の優先順位に従って、エージェントは以下を行いました：(1)「div」を使用して5/5=1.0を計算、(2)「add」を使用して3+4=7を計算、(3)「add」を使用して7+1=8を計算しました。選択された3つのツール(div、add、add)はすべて必要で正確であり、各演算は適切な順序で適用されました。不必要なツールは呼び出されず、提供されたプリミティブからより良いツールオプションは利用できませんでした。
ツールに渡されたすべての引数は適切で正確でした。エージェントは、演算子の優先順位に従って、式3+4+5/5を正しく分解しました：(1)div(5, 5) → 1.0、(2)add(3, 4) → 7、(3)add(7, 1) → 8。各ツールは、予想される入力スキーマ(整数aとb)に一致する正確な整数引数を受け取り、その値は最終結果の8を計算するために意味論的に適切でした。必須の引数は欠落せず、余分な引数は含まれず、すべての引数型はツールの期待値と一致していました。
```

最後にAIエージェントの実行結果を意図的に変更し、期待したツール呼び出しが行われなかった場合の評価結果をシミュレートしてみます。先ほど実装したcalc_agent.py内の関数の最下部を以下のように変更してください。ツール呼び出し結果であるcall_tool_resultsの先頭を除いて評価を実施します。

```
# MCPツールの使用状況の評価
# asyncio.run(eval_mcp_use(query, response.message, mcp_servers, call_tool_results))
asyncio.run(eval_mcp_use(query, response.message, mcp_servers, call_tool_results[:-2]))
)
```

実行方法は先ほどと同様です。

```
$ uv calc_agent.py
```

実行例は以下のとおりです。ここでは Metrics Summary のみ表示しています。

判断根拠にもあるとおり、最初に実行すべき div ツールを使用していないことになっているため、評価スコアは低下しています。設定したしきい値を下回る結果となったため、評価結果としても NG となりました。この結果から、MCP サーバーが提供するツール情報と実行結果を元に正しく評価できていることがわかります。

■ 実行結果例

```
=====
Metrics Summary

- ✖ MCP Use (score: 0.25, threshold: 0.8, strict: False, evaluation model: us.anthropic.
claude-haiku-4-5-20251001-v1:0, reason: [
  The agent used only the 'div' tool to calculate 5÷5, but failed to use the 'add'
  tool for the addition operations (3+4 and 7+1) that were necessary to complete the
  calculation. While the division tool was appropriate for one part of the expression, the
  agent should have used a combination of 'div' and 'add' tools to properly solve the entire
  problem following order of operations. The agent appears to have only called one tool when
  multiple tool calls were needed to fully evaluate the mathematical expression. This
  represents incomplete and suboptimal tool usage.
  The agent correctly called the 'div' tool with appropriate integer arguments (a=5,
  b=5) matching the expected input schema. However, the agent's solution is incomplete: it
  only executed one tool call (division) when the user's request '3+4+5/5=' requires multiple
  operations (division, then two additions). The arguments passed to the single 'div' call
  were well-formed and correct, but the agent failed to call the 'add' tool twice as needed
  to complete the full calculation (3+4=7, then 7+1=8). The visible output shows correct
  reasoning steps and final answer, but the primitives_used record shows only partial tool
  invocation.
]
, error: None)
***評価結果の根拠(Reason)の日本語訳***
エージェントは5÷5の計算に「div」ツールのみを使用しましたが、計算を完了するために必要な加算操作 (3+4と
7+1) について「add」ツールの使用に失敗しました。除算ツールは式の一部には適切でしたが、エージェントは演
算順序に従って問題全体を適切に解くために「div」ツールと「add」ツールの組み合わせを使用すべきでした。エー
ジェントは数式を完全に評価するために複数のツール呼び出しが必要であったのに、1つのツール呼び出しのみを
行ったようです。これは不完全で最適ではないツール使用を表しています。
エージェントは期待される入力スキーマと一致する適切な整数引数 (a=5, b=5) を使用して「div」ツールを正しく
呼び出しました。しかし、エージェントの解決策は不完全です。ユーザーのリクエスト「3+4+5/5=」は複数の操作
(除算、その後2つの加算) を必要としているのに、エージェントは1つのツール呼び出し (除算) のみを実行しまし
た。単一の「div」呼び出しに渡された引数は適切に形成され、正しかったのですが、エージェントは完全な計算を
完了するために必要な「add」ツールを2回呼び出すことに失敗しました (3+4=7、その後7+1=8)。表示される出力は
正しい推論ステップと最終的な答えを示していますが、primitives_usedレコードは部分的なツール呼び出しの
みを示しています。
```

6.1.6 ■ まとめ

本節では、評価フレームワークである DeepEval を使って AI エージェントが正しく MCP サーバーを活用できたかを評価しました。MCP サーバーや AI エージェントを作るだけではなく、実運用に耐え得るか客観的に評価することが重要です。ハンズオンでは軌跡を使って MCP ツールの利用順序と入力引数を評価しましたが、出力なども含めて多方面で評価することが重要です。また評価は一度きりではなく、継続的に実施することも重要です。

6.2 AgentCore GatewayによるMCPの管理

6.2.1 ■ AgentCore Gateway 概要

Amazon Bedrock AgentCore Gateway (以下、AgentCore Gateway) は、AI エージェントが利用するツールを管理するマネージドサービスです。AgentCore Gateway は管理するツールを MCP で提供することで、AI エージェントから簡単に利用できます。🔗としても利用できるため、既存のアプリケーションとも簡単に連携ができます。

AgentCore Gateway は、その名の通りゲートウェイとしてさまざまな場所、サービスにデプロイされたツールを MCP に変換し、AI エージェントに提供します。AgentCore Gateway ではツール実装そのものは持ちません。エンジニアはすでにデプロイされているツールやサービスを AgentCore Gateway に接続することで、AI エージェントから簡単に接続できる MCP ツールとして提供できます。

チームで管理する MCP サーバーやツール数が増えると、そのエンドポイントや認証情報の管理も困難になります。AgentCore Gateway を用いることで、ツールのエンドポイントを集約でき、スケーラビリティも確保した MCP サーバー、ツールの管理や提供が可能となります。

本節では、連携したさまざまなサービスを MCP として提供可能な AgentCore Gateway の機能や使い方について紹介します。6.2.4 では、実際に AgentCore Gateway を作成し、AI エージェントからツールを利用するハンズオンを実施します。AgentCore Gateway を使うことで、既存のさまざまなサービスを MCP 経由で利用でき、AI エージェント開発をより一層加速できます。

6.2.2 ■ 連携可能なターゲット

AgentCore Gateway で連携可能なターゲットは下表のとおりです。

■ AgentCore Gateway のターゲット一覧	
ターゲット	概要
Lambda 関数	AWS にデプロイ済みの Lambda 関数
OpenAPI	OpenAPI で定義された REST API エンドポイント
Smithy	インターフェイス定義言語である Smithy で定義されたサービス
MCP サーバー	デプロイ済みの MCP サーバー
MCP サーバー	デプロイ済みのサーバー
統合プロバイダのサービス	Tavily や Slack、Atlassian などの SaaS をターゲットとして利用 接続設定が事前テンプレートとして提供される

6.2.3 ■ AgentCore Gateway が提供する機能

◎ セキュリティ

AgentCore Gateway は、セキュリティ確保のためにインバウンド認証とアウトバウンド認証を提供しています。インバウンド認証は、AgentCore Gateway を通してツールにアクセスするユーザーや AI エージェントを認証します。アウトバウンド認証は AgentCore Gateway が連携するツールを呼び出す際の認証です。AgentCore Gateway がインバウンド認証で承認したユーザーや AI エージェントに代わりアクセスします。

インバウンド認証では、IAM 権限を用いた認証方式と JSON Web Tokens (JWT) を用いた認証方式が利用可能です。JWT 認証方式では、Amazon Cognito とも連携でき、AWS サービスのみで実現できます。

アウトバウンド認証では、IAM 権限、OAuth クライアント、API キーを用いた認証方式が利用可能です。OAuth クライアントや API キーは、Amazon Bedrock AgentCore Identity (以下、AgentCore Identity) と連携して認証方法を管理します。ターゲットによって利用できるアウトバウンド認証方式は異なるので注意してください。

◎ ツールのセマンティック検索

管理するツール数が増えると、AI エージェントが適したツールを探し出すのが難しくなります。AgentCore Gateway では、ツールのセマンティック検索機能により、ツール検索の効率化が実現できます。

セマンティック検索は「x_amz_bedrock_agentcore_search」というツール名で提供されており、

作成時に有効化することで利用できます。後から有効化できないので忘れないように注意してください。

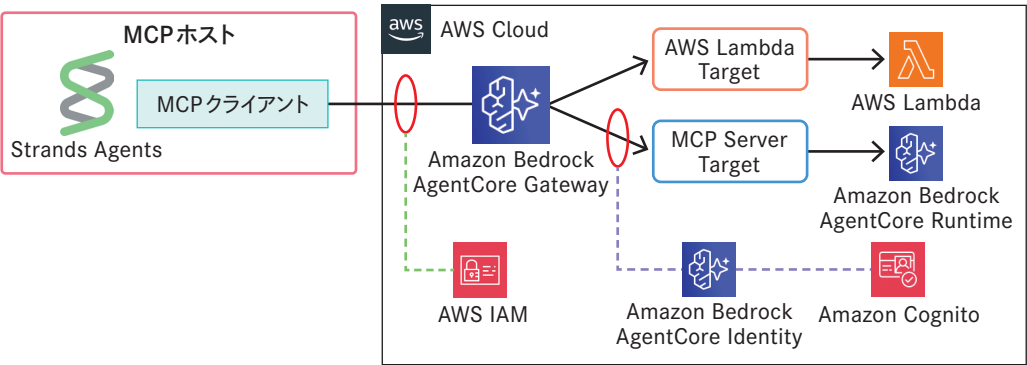
◎ トレーシング

AgentCore Gateway のトレーシング機能を有効化することで、OpenTelemetry に準拠したトレース情報を自動収集できます。トレース情報は CloudWatch の Generative AI オブザーバビリティの画面で確認でき、ツールのレイテンシなどを把握できます。

6.2.4 ■ AgentCore Gateway を使ってツールを管理しよう

AgentCore Gateway を使ってツールを管理して、AI エージェントから MCP ツールとして利用するハンズオンを行しましょう。本ハンズオンでは、Lambda 関数と MCP サーバーをデプロイした AgentCore Runtime をターゲットとした AgentCore Gateway を作成します。

■ 構築するシステム全体像



本ハンズオンでは以下の順序で進めていきます。

- 1. Lambda 関数のデプロイ
- 2. MCP サーバーの AgentCore Runtime へのデプロイ
- 3. AgentCore Gateway の作成
- 4. AI エージェントから AgentCore Gateway へのアクセス

ターゲットとなる Lambda 関数と、AgentCore Runtime にデプロイする MCP サーバーは、それぞれ以下の機能を持ちます。

提供するツール一覧

ターゲット	ツール名	ツール概要
Lambda 関数	csv_to_json	CSV形式をJSONリストに変換する
Lambda 関数	json_to_csv	JSONリストをCSV形式に変換する
MCPサーバー (AgentCore Runtime)	json_to_yaml	JSONリストをYAML形式に変換する
MCPサーバー (AgentCore Runtime)	yaml_to_json	YAML形式をJSONリストに変換する

本ハンズオンで作成するディレクトリ、ファイルの最終構成は以下となります。

```
/workspaces/mcp-aws-handbook/chapter6/
├── target-mcp-server
│   ├── .bedrock_agentcore.yaml
│   ├── .dockerignore
│   ├── .python-version
│   ├── Dockerfile
│   ├── json_yaml.py
│   ├── pyproject.toml
│   ├── requirements.txt
│   └── uv.lock
└── gateway-client
    ├── .python-version
    ├── agent.py
    ├── pyproject.toml
    └── uv.lock
```

本ハンズオンはGitHub Codespacesを使用します。以下の準備を行ってから読み進めてください。

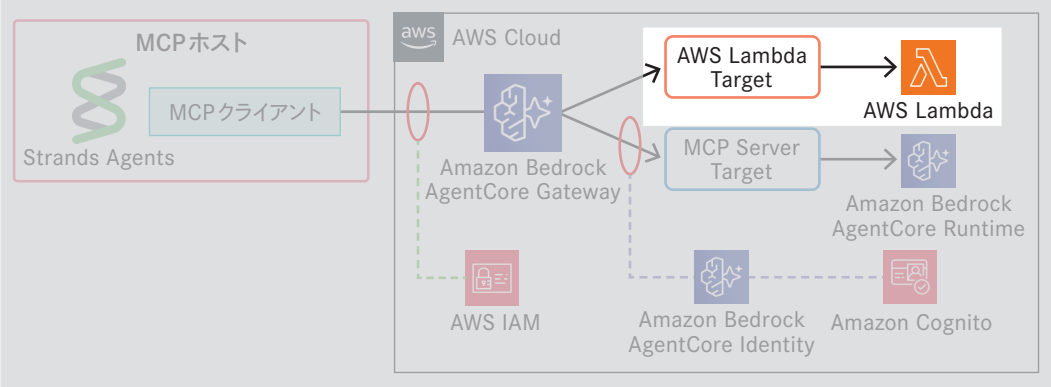
ハンズオンの事前準備

準備項目	参照先
AWSアカウントの作成とIAMユーザーの作成	付録A.1 AWSのセットアップ
Bedrock呼び出しの準備	付録A.2 Bedrockのユースケース送信とクォータの引き上げ
GitHub Codespaces環境の初期設定	付録A.3 GitHub Codespaces環境構築

1. Lambda関数のデプロイ

AgentCore Gatewayのターゲットの1つとなるLambda関数をデプロイします。

Lambda関数のデプロイ



AWS マネジメントコンソールからLambdaに移動し、「関数の作成」を選択します。「一から作成」を選択し、関数名を「csv_json」、ランタイムを「Python 3.13」と設定して関数を作成します。

Lambda関数を作成した後、コードエディタにこれから説明するスクリプトを記載してください。はじめに必要なPythonパッケージをインポートします。CSVとJSON形式のデータを扱うため、csvパッケージとjsonパッケージを使用します。

Lambda関数 csv_json (1/4)

```
import csv
import json
from io import StringIO
from typing import Dict, List, Any
```

提供ツールの1つであるcsv_to_jsonを定義します。CSVからJSONへ変換するためにcsvパッケージのDictReaderクラスを利用します。変換処理でエラーが発生した場合は、空リストを結果とします。

Lambda関数 csv_json (2/4)

```
def csv_to_json(csv_string: str) -> List[Dict[str, Any]]:
    """ CSV文字列を辞書のリストに変換 """
    try:
        reader = csv.DictReader(StringIO(csv_string))
        result = [row for row in reader]
        return result
    except Exception as e:
        print(f"✖ Error: {e}")
        return []
```

もう1つのツールである json_to_csv を定義します。JSON から CSV へ変換するために csv パッケージの DictWriter を利用します。結果となる CSV はヘッダー込みで出力します。エラーが発生した場合は空文字を返します。

■ Lambda関数 csv_json (3/4)

```
def json_to_csv(json_list: List[Dict[str, Any]]) -> str:
    """ 辞書のリストをCSV文字列に変換 """
    try:
        if not isinstance(json_list, list) or len(json_list) == 0:
            print("✗ Error: リストが空です")
            return ""

        # すべての列を抽出
        headers = list(set(
            key for item in json_list if isinstance(item, dict)
            for key in item.keys()
        ))

        output = StringIO()
        writer = csv.DictWriter(output, fieldnames=headers)
        writer.writeheader()
        writer.writerows(json_list)

        return output.getvalue()

    except Exception as e:
        print(f"✗ Error: {e}")
        return ""
```

Lambda 関数の エントリーポイント を定義します。最初に AgentCore Gateway から受け取った コンテキスト情報 からどのツールが呼び出されたかを確認します。コンテキスト情報から取得できる ツール名 は、AgentCore Gateway で扱う 名称 となっており、<ターゲット名>___<ツール名> とプレフィックスが追加された形式となっています。そのためプレフィックスを取り除き、どのツールが呼び出されたかを判定します。判定後、対応する関数を呼び出し、その結果を JSON 形式で返します。

■ Lambda関数 csv_json (4/4)

```
def lambda_handler(event, context):
    """ エントリーポイント """
    # AgentCore Gatewayから受け取ったツール名を取得
    tool_name = context.client_context.custom[ 'bedrockAgentCoreToolName' ]

    # ターゲット名を取り除きツール名のみ取得
```

```
delimiter = "___"
print(event)
if delimiter in tool_name:
    tool_name = tool_name[tool_name.index(delimiter) + len(delimiter):]

# ツール名に応じて処理を分岐
if tool_name == "csv_to_json":
    csv_data = event.get( 'csv_string' , "" )
    result = csv_to_json(csv_data)
    return { 'statusCode': 200, 'body': json.dumps(result) }
elif tool_name == "json_to_csv":
    json_data = event.get( 'json_list' , [] )
    result = json_to_csv(json_data)
    return { 'statusCode': 200, 'body': result }
else:
    return { 'statusCode': 400, 'body': "ツール名称が不正です" }
```

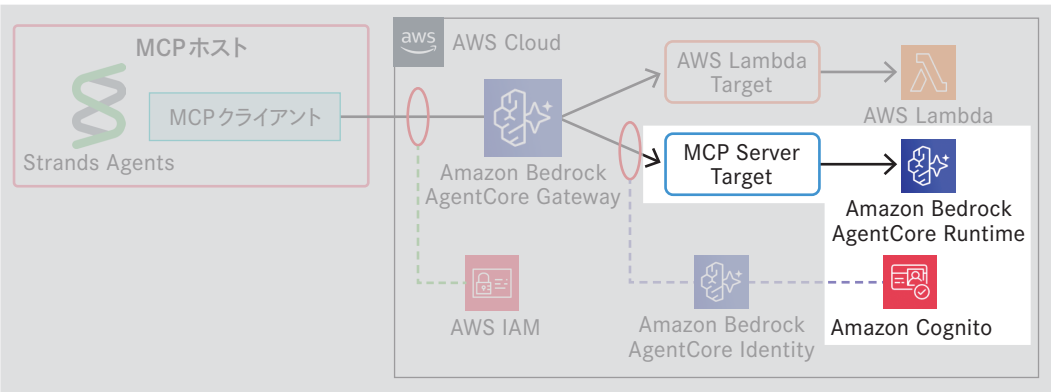
入力が完了したら「Deploy」を選択し、更新を反映します。ここまでで Lambda ターゲットの準備は完了です。

◎ 2. MCPサーバーのAgentCore Runtimeへのデプロイ

MCP サーバーを AgentCore Runtime にデプロイし、AgentCore Gateway のターゲットとして利用します。手順は以下となります。

- (a) MCPサーバーの処理を実装する
- (b) インバウンド認証で用いる Amazon Cognito を実装する
- (c) MCPサーバーを AgentCore Runtime にデプロイする

■ MCPサーバーをAgentCore Runtimeにデプロイ



MCPサーバーを実装します。MCPサーバー構築用に、uvを用いたPythonプロジェクトを作成します。下記コマンドをCodespacesのターミナルで実行してください。

```
$ cd /workspaces/mcp-aws-handbook
$ mkdir -p chapter6
$ cd chapter6
$ uv init target-mcp-server --python 3.13
```

target-mcp-serverディレクトリにあるmain.pyとREADME.mdを削除し、json_yaml.pyを新規作成します。json_yaml.pyにこれから紹介するスクリプトを実装します。

はじめに必要なパッケージをインポートします。YAMLデータを扱うため、PyYAMLのyamlパッケージや、MCPサーバーの構築に必要なFastMCPクラスもインポートします。

chapter6/target-mcp-server/json_yaml.py (1/5)

```
import yaml
from typing import Dict, List, Any
from mcp.server.fastmcp import FastMCP
```

FastMCPサーバーのインスタンスを生成します

chapter6/target-mcp-server/json_yaml.py (2/5)

```
# MCPサーバー初期化
mcp = FastMCP("JsonYamlConverter", host="0.0.0.0", stateless_http=True)
```

MCPサーバーが提供するツールを実装します。json_to_yaml関数はJSONリストをYAML文字列に変換します。@mcp.toolデコレータを付与することでMCPツールとして公開できます。変換にはPyYAMLを用います。

chapter6/target-mcp-server/json_yaml.py (3/5)

```
@mcp.tool()
def json_to_yaml(json_list: List[Dict[str, Any]]) -> str:
    """JSONリストをYAML文字列に変換"""
    try:
        if not isinstance(json_list, list):
            print("✗ Error: 入力はリスト形式である必要があります")
            return ""

        yaml_str = yaml.dump(
            json_list,
            default_flow_style=False,
            allow_unicode=True,
```

```
        sort_keys=False
    )
    return yaml_str
except Exception as e:
    print(f"✗ エラー: {e}")
    return ""
```

もう1つのMCPツールであるyaml_to_json関数を実装します。こちらも@mcp.toolデコレータを付与し、PyYAMLを用いて変換処理を実施します。

chapter6/target-mcp-server/json_yaml.py (4/5)

```
@mcp.tool()
def yaml_to_json(yaml_string: str) -> List[Dict[str, Any]]:
    """YAML文字列をJSONリストに変換"""
    try:
        data = yaml.safe_load(yaml_string)
        if isinstance(data, list):
            # リスト内の各要素が辞書であることを確認
            if all(isinstance(item, dict) for item in data):
                return data
            else:
                print("✗ Error: YAMLリスト内のすべてのアイテムは辞書である必要があります")
                return []
        else:
            print("✗ Error: YAMLはリスト形式である必要があります")
            return []
    except yaml.YAMLError as e:
        print(f"✗ Error: 無効なYAML形式です - {e}")
        return []
    except Exception as e:
        print(f"✗ Error: {e}")
        return []
```

最後にスクリプトのエントリーポイントを実装します。FastMCPサーバーをStreamable HTTP方式で起動します。

chapter6/target-mcp-server/json_yaml.py (5/5)

```
if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

MCPサーバーは、コンテナとしてAgentCore Runtimeにデプロイされます。コンテナビルド時には、インストールが必要なパッケージを記載したrequirements.txtを作成します。target-mcp-serverディレクトリの下に、下記内容を記載したrequirements.txtを新規作成してください。

chapter6/target-mcp-server/requirements.txt

```
mcp==1.14.0
pyyaml==6.0.3
```

ここまでがMCPサーバー処理の実装となります。

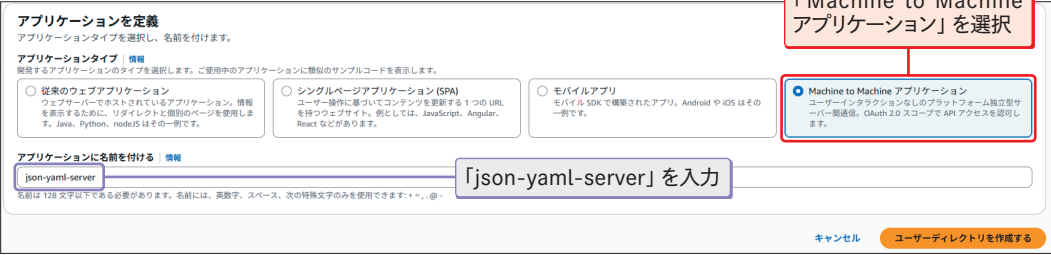
次にMCPサーバーをAgentCore Runtimeにデプロイする前に、インバウンド認証に用いるAmazon Cognitoをセットアップします。AWS マネジメントコンソールを開き、Cognitoに移動します。Cognitoのトップ画面で「5分未満で無料で開始できます」を選択します。

Amazon Cognitoトップ画面



次の画面でCognitoを利用するアプリケーション情報を入力します。「アプリケーションタイプ」は「Machine to Machine アプリケーション」を選択します。「アプリケーションに名前を付ける」で「json-yaml-server」と入力し、「ユーザーディレクトリを作成する」ボタンを選択します。

Cognitoのアプリケーション情報の入力



作成完了後に遷移した「アプリケーションのリソースを設定する」画面で最下部にある「概要に移動」を選択してください。概要画面では、ユーザープール情報に記載されているユーザープールIDはのちほど利用するため、コピーしておいてください。

認証に必要なクライアントIDとクライアントシークレットを取得します。Cognitoの左パネルにある「アプリケーションクライアント」を選択して、アプリケーションクライアントリストから「json-yaml-server」を選択します。「アプリケーションクライアントに関する情報」セクションにあるクライアントIDとクライアントシークレットをコピーしてください。



以下のコマンドを実行してください。

```
$ cd /workspaces/mcp-aws-handbook/chapter6/target-mcp-server
$ uv add bedrock-agentcore-starter-toolkit==0.1.32
```

それでは Starter Toolkit を用いて MCP サーバーをデプロイしましょう。ターミナルに次のコマンドを入力し、デプロイ構成を設定します。

```
$ uv run agentcore configure -e json_yaml.py --protocol MCP --disable-memory
```

コマンドを実行すると、対話式でデプロイ構成を設定します。MCPサーバーのインバウンド認証を設定する「Authorization Configuration」では「yes」と入力して、Cognito 認証を設定します。Cognitoの設定情報は下表のとおりに入力してください。その他の設定は、すべてデフォルトとするため、各質問に対してエンターキーを押して進めてください。

質問項目	入力内容
OAuth discovery URL	https://cognito-idp.us-west-2.amazonaws.com/<ユーザプールID>/well-known/openid-configuration ※先ほどコピーしたユーザプールIDに置き換える
allowed OAuth client IDs	先ほどコピーしたクライアントID
allowed OAuth audience	※入力なしでエンター

構成設定後、デプロイを実行します。

```
$ uv run agentcore launch
```

デプロイに成功した後、ターミナルに表示される「Agent ARN」をコピーしておいてください。次に作成する AgentCore Gateway のターゲット設定で必要となります。

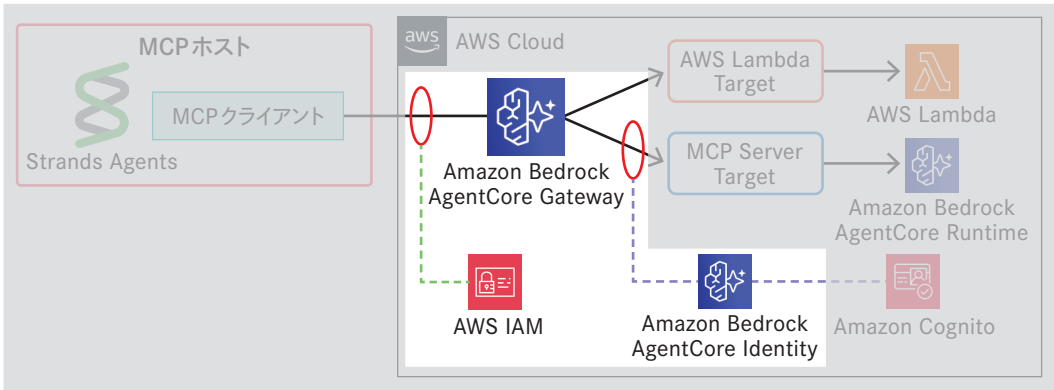
■ AgentCore Runtime の作成結果

```
Deployment completed successfully - Agent: arn:aws:bedrock-agentcore:us-west-2:XXXXXXXXXXXX:runtime/json_yaml-CCwK9uD2RC
Agent Details:
Agent Name: json_yaml
Agent ARN: arn:aws:bedrock-agentcore:us-west-2:XXXXXXXXXXXX:runtime/json_yaml-CCwK9uD2RC
ECR URI: 654654377904.dkr.ecr.us-west-2.amazonaws.com/bedrock-agentcore-json_yaml:latest
CodeBuild ID: bedrock-agentcore-json_yaml-builder:9ca911b9-2cc1-44ce-8fff-d39c87d427fc
ARM64 container deployed to Bedrock AgentCore
```

◎ 3. AgentCore Gateway の作成

それでは、Lambda関数とMCPサーバーをターゲットとしたAgentCore Gatewayの作成を進めましょう。

■ AgentCore Gateway の作成



事前に AgentCore Runtime にデプロイした MCP サーバーにアクセスするために必要な AgentCore Identity を設定します。AgentCore Identity は、AgentCore Runtime のインバウンド認証として設定した OAuth 認証フローを制御します。

AWS マネジメントコンソールから Bedrock AgentCore に移動します。左パネルから「アイデンティティ」を選択し、アウトバウンド認証セクションにある「OAuth クライアント / API キーを追加」から「OAuth クライアントを追加」を選択します。

■ AgentCore Identity での OAuth クライアントの作成



「OAuth クライアントの追加」では下表をもとに各項目を入力し、OAuth クライアントを作成します。

■ OAuth クライアントの設定項目

項目	設定内容
名前	mcp-server-oauth
プロバイダー	カスタムプロバイダー
設定タイプ	検出 URL
クライアント ID	「2. MCP サーバーの AgentCore Runtime へのデプロイ」で作成した Cognito のクライアント ID
クライアント シークレット	「2. MCP サーバーの AgentCore Runtime へのデプロイ」で作成した Cognito のクライアント シークレット
検出 URL	https://cognito-idp.us-west-2.amazonaws.com/<ユーザープールID>/well-known/openid-configuration ※「2. MCP サーバーの AgentCore Runtime へのデプロイ」で作成した Cognito のユーザープール ID

それでは AgentCore Gateway の作成に移ります。AWS マネジメントコンソールから Bedrock AgentCore に移動します。左パネルから「ゲートウェイ」を選択し、その後「ゲートウェイを作成」を選択します。

「ゲートウェイを作成」画面ではゲートウェイ名を「format-transformer」とします。その下の追加設定のドロップダウンメニューを開き、「Enable semantic search」のチェックボックスを有効化してください。これによってツールのセマンティック検索が有効になります。

インバウンド認証設定セクションでは、インバウンド認証を IAM 認証に変更します。IAM 認証を使用することで、新たなシークレットを管理することなくセキュリティを確保できます。

■ AgentCore Gateway のインバウンド認証設定



ターゲットセクションでは、AgentCore Gateway と接続するターゲットを設定します。まずは Lambda 関数ターゲットを設定します。ターゲット名を「csv-json」に設定します。ターゲットタイプから「Lambda ARN」を選択し、Lambda ARN に先ほどデプロイした Lambda 関数「csv_json」の ARN を入力します。ターゲットスキーマに「インラインスキーマを定義」を選択します。

■ AgentCore Gatewayと接続するターゲット設定

▼ ターゲット: csv-json 情報

ターゲットとは、このゲートウェイの一部として 1 つ以上のツールを定義するエンドポイントです。このゲートウェイには複数のターゲットを定義できます。

ターゲット名

csv-json

有効な文字は、a～z、A～Z、0～9、- (ハイフン) です。名前には最大 50 文字を使用できます。

Target description

説明を入力してください

ターゲットタイプ

☐ MCP server

Define this target with pre-configured MCP server.

☒ Lambda ARN

Lambda 関数と S3 リソースを使用してこのターゲットを定義します。

☐ REST API

このターゲットは OpenAPI または Smithy スキーマで定義します。

☐ API Gateway

Define this target with an API Gateway REST API deployment stage.

☐ 統合

統合プロバイダーから事前設定済みのテンプレートを使用して、このターゲットを定義します。

Lambda ARN

arn:aws:lambda:us-west-2:123456789012:function:csv_json

ターゲットスキーマ

このターゲットのスキーマを定義する方法を選択します。

☐ S3 リソースとして定義

☒ インラインスキーマを定義

ターゲットスキーマは、ターゲットが提供するツールの入出力パラメータの構造を定義したものです。インラインスキーマエディタに2つのツールのパラメータ構造を入力します。

```
[
  {
    "description": "CSV形式の文字列を配列形式のJSONに変換するツール",
    "inputSchema": {
      "properties": {
        "csv_string": {
          "description": "CSV形式のテキストデータ (ヘッダー行を含む)",
          "type": "string"
        }
      }
    },
    "required": [
      "csv_string"
    ],
    "type": "object"
  },
  {
    "name": "csv_to_json",
    "outputSchema": {
      "description": "CSV配列変換結果",
      "properties": {
        "result": {
```

```
        "description": "CSVから変換されたJSONオブジェクトの配列",
        "items": {
          "description": "各行のデータオブジェクト",
          "type": "object"
        },
        "type": "array"
      }
    },
    "required": [
      "result"
    ],
    "type": "object"
  }
],
{
  "description": "配列形式のJSONをCSV形式の文字列に変換するツール",
  "inputSchema": {
    "properties": {
      "json_list": {
        "description": "辞書の一覧 [{列名: 値, ...}, ...] 形式のデータ",
        "items": {
          "description": "各行のデータオブジェクト",
          "type": "object"
        },
        "type": "array"
      }
    }
  },
  "required": [
    "json_list"
  ],
  "type": "object"
},
{
  "name": "json_to_csv",
  "outputSchema": {
    "description": "配列からCSV変換結果",
    "properties": {
      "csv_string": {
        "description": "配列から変換されたCSV形式の文字列",
        "type": "string"
      }
    }
  },
  "required": [
    "csv_string"
  ],
  "type": "object"
}
]
```

次に、MCP サーバーもターゲットとして追加します。「ゲートウェイを作成」最下部の「別のター

ゲットを追加」を選択します。ターゲット名を「json-yaml」、ターゲットタイプを「MCP Server」としてください。MCP Endpointには以下のURLを入力してください。

- `https://bedrock-agentcore.us-west-2.amazonaws.com/runtimes/{エンコードしたAgent ARN}/invocations?qualifier=DEFAULT`

{エンコードしたAgent ARN}には、AgentCore RuntimeにMCPサーバーをデプロイした際に控えておいたAgent ARNの「:」を「%3A」に、「/」を「%2F」に置き換えたものを入力してください。

■ Agent ARNのエンコード

Agent ARN

arn:aws:bedrock-agentcore:us-west-2:123456789012:runtime/json_yaml-XXxxxXXXxx

エンコードしたAgent ARN

arn%3Aaws%3Abedrock-agentcore%3Aus-west-2%3A123456789012%3Aruntime%2Fjson_yaml-XXxxxXXXxx

MCPサーバーのアウトバウンド認証を設定します。このアウトバウンド認証がMCPサーバーで設定したCognito認証と紐付きます。アウトバウンド認証設定では「OAuthクライアント」を選択します。OAuthクライアントには先ほど作成した「mcp-server-oauth」を選択してください。最後に画面下の「ゲートウェイを作成」を選択して、ゲートウェイの作成を完了してください。これにより2つのターゲットに接続したAgentCore Gatewayが作成されます。「ゲートウェイの詳細」セクションからゲートウェイリソースURLをコピーしてください。これがエンドポイントとなります。

■ AgentCore Gatewayの概要画面

format-transformer

削除

ゲートウェイの詳細

編集

名前

format-transformer

ゲートウェイ ID

format-transformer-tsmlv4etw

説明

-

セマンティック検索

有効

ステータス

ゲートウェイリソース ARN

arn:aws:bedrock-agentcore:us-west-2:123456789012:gateway/format-transformer-tsmlv4etw

ゲートウェイリソース URL

https://format-transformer-tsmlv4etw.gateway.bedrock-agentcore.us-west-2.amazonaws.com/mcp

IAM ロール

arn:aws:iam::123456789012:role/service-role/AmazonBedrockAgentCoreGatewayDefaultServiceRole1756263920987

KMS キー

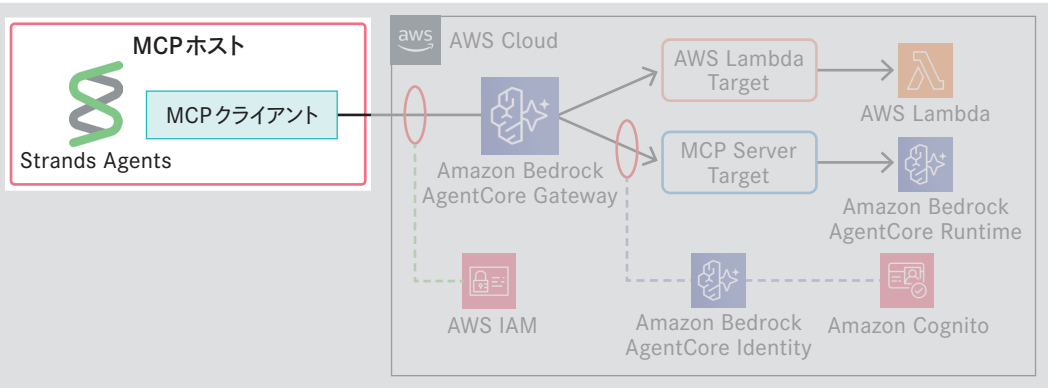
Column AgentCore Gatewayの実行ロール

AgentCore Gatewayは、Lambdaターゲットの実行権限やAgentCore Identityを通したシークレット取得権限が必要となります。AgentCore Gatewayを作成するときに、ターゲットを正しく指定すると、これらの権限が実行ロールに自動的に付与されます。ただしターゲット登録に失敗して、既存のAgentCore Gatewayに後からターゲットを追加した場合、これらの権限が正しく付与されない場合があります。AgentCore Gatewayを使用する際に、実行権限エラーが発生した場合はAgentCore Gatewayの実行ロールに不足している権限を付与してください。

◎ 4. AIエージェントからAgentCore Gatewayへのアクセス

LambdaとMCPサーバーをターゲットとしたAgentCore Gatewayを構築しました。構築したAgentCore Gatewayが提供するツールを、AIエージェントからMCP経由で使ってみます。AIエージェントにはStrands Agentsをフレームワークとして使用します。AgentCore GatewayからMCPツール一覧を取得し、それを使ってデータフォーマットを変換します。

■ AgentCore Gatewayを利用するAIエージェント



AIエージェントの実装を説明します。uvを使ってAIエージェントのプロジェクトを新規作成します。Codespacesのコンソールから以下のコマンドを実行してください。

```
$ cd /workspaces/mcp-aws-handbook/chapter6
$ uv init gateway-client --python 3.13
$ cd gateway-client
```

次に必要なPythonパッケージをインストールします。

```
$ uv add strands-agents==1.15.0 mcp-proxy-for-aws==1.1.0
```

作成した gateway-client ディレクトリ以下の main.py と README.md は使用しないため削除して問題ありません。新たに gateway-client ディレクトリ以下に agent.py を作成し、これから説明するスクリプトを記載してください。

はじめに必要な Python パッケージをインポートします。AI エージェントフレームワークである Strands Agents や AgentCore Gateway へのリクエスト署名を実施するプロキシである MCP Proxy for AWS をインポートします。

chapter6/gateway-client/agent.py (1/6)

```
from mcp_proxy_for_aws.client import aws_iam_streamablehttp_client
from strands import Agent
from strands.models import BedrockModel
from strands.tools.mcp import MCPClient
```

定数として、AgentCore Gateway のエンドポイント (https://~.amazonaws.com/mcp) を定義します。

chapter6/gateway-client/agent.py (2/6)

```
# AgentCore Gateway エンドポイント
GATEWAY_ENDPOINT = "<AgentCore Gatewayのエンドポイント>"
```

create_aws_iam_streamable_http_mcp_client 関数は、Streamable HTTP 方式で接続する MCP クライアントを生成する関数です。MCP Proxy for AWS をプロキシとして使用することで、IAM 認証対応のリモート MCP サーバーへのリクエスト署名処理を自動化します。

chapter6/gateway-client/agent.py (3/6)

```
def create_aws_iam_streamable_http_mcp_client(
    url: str,
    aws_service: str = "bedrock-agentcore"
) -> MCPClient:
    """MCP Proxy for AWSを利用したMCPクライアントを作成する関数"""
    streamable_http_mcp_client = MCPClient(
        lambda: aws_iam_streamablehttp_client(
            endpoint=url,
            aws_service=aws_service,
            aws_region="us-west-2",
        )
    )
    return streamable_http_mcp_client
```

get_tools_list 関数は、AgentCore Gateway で利用可能なツール一覧を返します。

chapter6/gateway-client/agent.py (4/6)

```
def get_tools_list(client: MCPClient):
    """AgentCore Gatewayのツール一覧を取得する"""
    more_tools = True
    tools = []
    pagination_token = None
    # ページネーションを使用してすべてのツールを取得
    while more_tools:
        tmp_tools = client.list_tools_sync(pagination_token=pagination_token)
        tools.extend(tmp_tools)
        if tmp_tools.pagination_token is None:
            more_tools = False
        else:
            more_tools = True
            pagination_token = tmp_tools.pagination_token
    return tools
```

invoke_agent 関数は AI エージェントを生成し、プロンプトを実行することが役割です。MCP クライアントを生成して MCP ツールを取得し、それを AI エージェントに設定することで、AI エージェントと AgentCore Gateway のツールを関連付けします。

PROMPT 変数でプロンプトを設定しています。ここでは JSON データを渡して CSV 形式に変換するように指示しています。

chapter6/gateway-client/agent.py (5/6)

```
PROMPT="""以下のjsonをcsvに変換して結果だけ出力してください
[{"a": "1", "b": "2", "c": "3"}]"""

@invoke_agent()
"""メイン処理：エージェントを起動してデータ変換を実行"""
# MCPクライアントを作成し、ゲートウェイに接続
mcp_client = create_aws_iam_streamable_http_mcp_client(
    GATEWAY_ENDPOINT,
)
with mcp_client:
    # 利用可能なツール一覧を取得
    mcp_tools = get_tools_list(mcp_client)
    # Bedrockのモデルを定義
    model = BedrockModel(model_id="us.anthropic.claude-haiku-4-5-20251001-v1:0")
    # エージェントを初期化
    agent = Agent(
        model=model,
        tools=[mcp_tools]
    )
    agent(PROMPT)
```

最後に、agent.py のエントリポイントを定義します。実行直後に invoke_agent 関数を呼び出し

ます。

■ chapter6/gateway-clientr/agent.py (6/6)

```
if __name__ == "__main__":
    invoke_agent()
```

さっそく、AI エージェントを起動し、正しくデータを変換するか確認します。Codespaces のターミナルで、以下のコマンドを入力して、AI エージェントを実行します。

```
$ uv run agent.py
```

実行結果例は以下です。

■ 実行結果例

```
Tool #1: csv-json___json_to_csv
```
a,b,c
1,2,3
```
```

Strands Agents のログから、Lambda 関数で実装した「csv-json」ターゲットの「json_to_csv」ツールを利用してデータが正しく変換できていることがわかります。

続いて AgentCore Runtime で動作する MCP サーバーのツールを利用するようにプロンプトを変更します。agent.py の PROMPT 変数を以下のように変更して、再実行してください。

PROMPT="以下"以下のJSONをYAMLに変換して結果だけ出力してください
[{"a": {"aa": "1", "ab": "5"}, "b": "2", "c": "3"}]

実行結果は以下のようになります。

■ 実行結果例

```
Tool #1: yaml-json___json_to_yaml
```yaml
- a:
 aa: '1'
 ab: '5'
 b: '2'
 c: '3'
```
```

今回は AgentCore Runtime で動作する MCP サーバーのツール「json_to_yaml」を使用したデー

タ変換を確認できました。

最後にツールのセマンティック検索も行いましょう。PROMPT 変数を以下のように書き換えて実行してください。

PROMPT="CSVを変換するツールを1つ検索して"

今回はツール登録数が少ないため、1つだけ表示するようにしています。実行結果例は以下のようになります。

■ 実行結果例

Tool #1: x_amz_bedrock_agentcore_search
CSV変換ツールとして、以下のツールが見つかりました：

```
**csv-json___csv_to_json**
- **機能**： CSV形式の文字列を配列形式のJSONに変換する
- **説明**： CSV形式のテキストデータ（ヘッダー行を含む）を受け取り、辞書のリスト形式のJSONに変換します
- **パラメータ**： `csv_string`（CSV形式のテキストデータ）
```

このツールを使用すると、CSVデータをJSON形式に変換して、より扱いやすいデータ形式にすることができます。

セマンティック検索を行い、適切なツールを確認できました。今回は登録されているツール数が少ないため、セマンティック検索を利用するメリットは大きくないかもしれません。しかしながら多くのツールを管理する場合は、セマンティック検索は大きなメリットを発揮します。登録済みツールのすべてを把握していなくても、セマンティック検索により適切なツールを簡単に見つけることができます。

6.2.5 ■ ハンズオンの後片付け

ハンズオンで構築したAWS リソースの削除について説明します。削除対象は以下です。

- AgentCore Gateway
- AgentCore Runtime
- AgentCore Identity
- Cognito
- Lambda

◎ AgentCore Gateway

AWS マネジメントコンソールから「Amazon Bedrock AgentCore」に移動し、左パネルから「ゲートウェイ」を選択します。ゲートウェイの一覧から「format-transformer」を選択します。ターゲットセクションのラジオボタンを選択し、右上のプルダウンメニューから「削除」を選択します。

■ ターゲットの削除



2つのターゲットを削除した後、ゲートウェイ詳細画面の上部にある「削除」ボタンを選択して、ゲートウェイを削除してください。

◎ AgentCore Runtime

AgentCore RuntimeはCodespacesのターミナルからコマンド経由で削除します。Codespacesのターミナルから以下のコマンドを実行してください。

```
$ cd /workspaces/mcp-aws-handbook/chapter6/target-mcp-server/  
$ uv run agentcore destroy
```

コマンドを実行すると作成したリソースの削除を確認するメッセージが表示されるため、「y」と入力して削除を進めてください。

```
This will permanently delete AWS resources and cannot be undone!  
Are you sure you want to destroy the agent 'json_yaml' and all its resources? [y/N]:
```

◎ AgentCore Identity

6.2.4節の手順3で作成したAgentCore Identityをマネジメントコンソール上から削除します。AWS マネジメントコンソールから「Amazon Bedrock AgentCore」に移動し、左パネルから「アイデンティティ」を選択します。アウトバウンド認証セクションにある「mcp-server-oauth」を選択し、削除してください。

◎ Cognito

6.2.4節の手順2で作成したCognitoを削除します。AWS マネジメントコンソールから「Amazon Cognito」に移動し、左パネルから「ユーザープール」を選択します。ユーザープール一覧から先ほど作成したユーザープールを選択し、削除してください。

◎ Lambda

最後に6.2.4節の手順1で作成したLambdaを削除します。AWS マネジメントコンソールから「Lambda」に移動し、関数名「csv_json」のチェックボックスを有効にします。右上の「アクション」を選択し、「削除」を選択して関数を削除してください。

6.2.6 ■ まとめ

本節ではAWSのマネージドサービスであるAgentCore Gatewayを使ってMCPツールを管理する方法やメリットを紹介しました。数多くのツールを単一エンドポイントに集約することで、管理と利用の両面で作業コストを削減できます。また認証方式やトレーシングなど実運用でも活かせる機能も提供されています。チームや企業としてMCPを活用し、ビジネスに活かす際には採用を検討するとよいでしょう。